

Load Datasets

```
In [3]: from datasets import load_dataset
```

```
In [6]: # Test dataset Loading
try:
    commonsense_qa = load_dataset("commonsense_qa")
    print("Dataset structure:", commonsense_qa)

    # Look at examples
    for i in range(3):
        example = commonsense_qa['train'][i]
        print(f"Question: {example['question']}")
        print(f"Choices: {example['choices']}")
        print(f"Answer: {example['answerKey']}")
        print("-" * 50)
except Exception as e:
    print(f"❌ Dataset error: {e}")
```

```
Dataset structure: DatasetDict({
  train: Dataset({
    features: ['id', 'question', 'question_concept', 'choices', 'answerKey'],
    num_rows: 9741
  })
  validation: Dataset({
    features: ['id', 'question', 'question_concept', 'choices', 'answerKey'],
    num_rows: 1221
  })
  test: Dataset({
    features: ['id', 'question', 'question_concept', 'choices', 'answerKey'],
    num_rows: 1140
  })
})
Question: The sanctions against the school were a punishing blow, and they seemed to
what the efforts the school had made to change?
Choices: {'label': ['A', 'B', 'C', 'D', 'E'], 'text': ['ignore', 'enforce', 'authori
tarian', 'yell at', 'avoid']}
Answer: A
-----
Question: Sammy wanted to go to where the people were. Where might he go?
Choices: {'label': ['A', 'B', 'C', 'D', 'E'], 'text': ['race track', 'populated area
s', 'the desert', 'apartment', 'roadblock']}
Answer: B
-----
Question: To locate a choker not located in a jewelry box or boutique where would yo
u go?
Choices: {'label': ['A', 'B', 'C', 'D', 'E'], 'text': ['jewelry store', 'neck', 'jew
elry box', 'jewelry box', 'boutique']}
Answer: A
-----
```

```
In [7]: # Use the basic SNLI dataset to start
snli = load_dataset("stanfordnlp/snli")
```

```
print(snli['train'][0])
```

```
# You can generate explanations for these examples
# This actually aligns with your research - creating better explanations!
```

```
{'premise': 'A person on a horse jumps over a broken down airplane.', 'hypothesis':
'A person is training his horse for a competition.', 'label': 1}
```

```
In [15]: # Look at the dataset structure
print("CommonsenseQA structure:")
print(commonsense_qa)
print("\nSample CommonsenseQA example:")
example = commonsense_qa['train'][0]
for key, value in example.items():
    print(f"{key}: {value}")
```

CommonsenseQA structure:

```
DatasetDict({
  train: Dataset({
    features: ['id', 'question', 'question_concept', 'choices', 'answerKey'],
    num_rows: 9741
  })
  validation: Dataset({
    features: ['id', 'question', 'question_concept', 'choices', 'answerKey'],
    num_rows: 1221
  })
  test: Dataset({
    features: ['id', 'question', 'question_concept', 'choices', 'answerKey'],
    num_rows: 1140
  })
})
```

Sample CommonsenseQA example:

id: 075e483d21c29a511267ef62bedc0461

question: The sanctions against the school were a punishing blow, and they seemed to what the efforts the school had made to change?

question_concept: punishing

choices: {'label': ['A', 'B', 'C', 'D', 'E'], 'text': ['ignore', 'enforce', 'authoritarian', 'yell at', 'avoid']}

answerKey: A

```
In [16]: # Look at SNLI structure
print("\nSNLI structure:")
print(snli)
print("\nSample SNLI example:")
example = snli['train'][0]
for key, value in example.items():
    print(f"{key}: {value}")
```

SNLI structure:

```
DatasetDict({
  test: Dataset({
    features: ['premise', 'hypothesis', 'label'],
    num_rows: 10000
  })
  validation: Dataset({
    features: ['premise', 'hypothesis', 'label'],
    num_rows: 10000
  })
  train: Dataset({
    features: ['premise', 'hypothesis', 'label'],
    num_rows: 550152
  })
})
```

Sample SNLI example:

premise: A person on a horse jumps over a broken down airplane.

hypothesis: A person is training his horse for a competition.

label: 1

Sample the datasets

```
In [19]: import random

# --- CommonsenseQA random selection ---
num_csqa = random.randint(15, 20)
csqa_indices = random.sample(range(len(commonsense_qa['train'])), num_csqa)
commonsense_examples = []
for i in csqa_indices:
    example = commonsense_qa['train'][i]
    commonsense_examples.append({
        'id': i,
        'question': example['question'],
        'choices': example['choices']['text'],
        'answer': example['answerKey'],
        'dataset': 'commonsense_qa'
    })

# --- SNLI random selection (only valid label) ---
# Gather all valid indices first
valid_snli_indices = [i for i, ex in enumerate(snli['train']) if ex['label'] != -1]
num_snli = random.randint(15, 20)
snli_indices = random.sample(valid_snli_indices, num_snli)
snli_examples = []
for i in snli_indices:
    example = snli['train'][i]
    snli_examples.append({
        'id': i,
        'premise': example['premise'],
        'hypothesis': example['hypothesis'],
        'label': example['label'],
        'dataset': 'snli'
    })
```

```
# --- Combine both ---
all_examples = commonsense_examples + snli_examples
```

```
In [20]: # --- Write to file ---
label_names = ['entailment', 'neutral', 'contradiction']

with open("working examples.txt", "w", encoding="utf-8") as f:
    f.write("=== CommonsenseQA Examples ===\n\n")
    for ex in commonsense_examples:
        f.write(f"ID: {ex['id']}\n")
        f.write(f"Question: {ex['question']}\n")
        f.write(f"Choices: {ex['choices']}\n")
        f.write(f"Answer: {ex['answer']}\n")
        f.write("-" * 60 + "\n")
    f.write("\n=== SNLI Examples ===\n\n")
    for ex in snli_examples:
        f.write(f"ID: {ex['id']}\n")
        f.write(f"Premise: {ex['premise']}\n")
        f.write(f"Hypothesis: {ex['hypothesis']}\n")
        f.write(f"Label: {label_names[ex['label']]}\n")
        f.write("-" * 60 + "\n")

print("✅ File 'working examples.txt' written successfully.")
```

✅ File 'working examples.txt' written successfully.

Load Model

```
In [22]: from transformers import RobertaTokenizer, RobertaForMultipleChoice
import torch

# Load model & tokenizer
model_name = "roberta-base"
tokenizer = RobertaTokenizer.from_pretrained(model_name)
model = RobertaForMultipleChoice.from_pretrained(model_name)

# Move model to GPU if available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

# Format a CommonsenseQA example
def format_commonsense_example(question, choices):
    # Each choice is paired with the same question
    encoded = tokenizer(
        [question] * len(choices),
        choices,
        return_tensors="pt",
        padding=True,
        truncation=True,
    )
    return encoded

# Example input
question = "What sort of wet area does an anemone inhabit?"
choices = ['wet pool', 'tidal basin', 'vase of flowers', 'tide pool', 'field of flo
```

```

# Prepare input for the model
inputs = format_commonsense_example(question, choices)
inputs = {k: v.unsqueeze(0).to(device) for k, v in inputs.items()} # Add batch dim

# Predict
with torch.no_grad():
    outputs = model(**inputs)
    logits = outputs.logits
    predicted_idx = torch.argmax(logits, dim=1).item()

print(f"Question: {question}")
print("Choices:")
for idx, choice in enumerate(choices):
    print(f"{idx}: {choice}")
print(f"\nModel Prediction: {choices[predicted_idx]} (index {predicted_idx})")

```

Some weights of RobertaForMultipleChoice were not initialized from the model checkpoint at roberta-base and are newly initialized: ['classifier.bias', 'classifier.weight', 'roberta.pooler.dense.bias', 'roberta.pooler.dense.weight']

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

c:\Users\Work\anaconda3\envs\explainable_ai\Lib\site-packages\torch\nn\modules\module.py:1762: FutureWarning: `encoder\_attention\_mask` is deprecated and will be removed in version 4.55.0 for `RobertaSdpaSelfAttention.forward`.

```
return forward_call(*args, **kwargs)
```

Question: What sort of wet area does an anemone inhabit?

Choices:

- 0: wet pool
- 1: tidal basin
- 2: vase of flowers
- 3: tide pool
- 4: field of flowers

Model Prediction: field of flowers (index 4)

```

In [ ]: from lime.lime_text import LimeTextExplainer
import numpy as np

class CommonsenseQAPredictor:
    def __init__(self, model, tokenizer):
        self.model = model
        self.tokenizer = tokenizer

    def predict_proba(self, texts):
        # LIME expects this format
        # Convert texts to model inputs and return probabilities
        results = []
        for text in texts:
            # Your prediction logic here
            # Return probability distribution over choices
            pass
        return np.array(results)

# Initialize LIME
explainer = LimeTextExplainer(class_names=['A', 'B', 'C', 'D', 'E'])

```

```

predictor = CommonsenseQAPredictor(model, tokenizer)

# Generate explanation for your first example
explanation = explainer.explain_instance(
    question + " " + " ".join(choices),
    predictor.predict_proba,
    num_features=10
)

```

```

In [27]: from transformers import BertTokenizer, BertForMultipleChoice, Trainer, TrainingArg
import torch
from torch.utils.data import Dataset

class CommonsenseQADataset(Dataset):
    def __init__(self, data, tokenizer, max_length=64):
        self.data = data
        self.tokenizer = tokenizer
        self.max_length = max_length

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        item = self.data[idx]
        question = item["question"]
        choices = item["choices"]
        answer = item["answer"]

        # Tokenize each (question, choice)
        encoding = self.tokenizer(
            [question] * len(choices),
            choices,
            padding='max_length',
            truncation=True,
            max_length=self.max_length,
            return_tensors="pt"
        )
        return {
            "input_ids": encoding["input_ids"],
            "attention_mask": encoding["attention_mask"],
            "labels": torch.tensor(answer)
        }

```

```

In [24]: # Prepare your data
examples = commonsense_examples

model_name = "bert-base-uncased"
tokenizer = BertTokenizer.from_pretrained(model_name)
model = BertForMultipleChoice.from_pretrained(model_name)

# Dataset
dataset = CommonsenseQADataset(examples, tokenizer)

```

```
tokenizer_config.json: 0%|          | 0.00/48.0 [00:00<?, ?B/s]
```

c:\Users\Work\anaconda3\envs\explainable_ai\Lib\site-packages\huggingface_hub\file_download.py:143: UserWarning: `huggingface\_hub` cache-system uses symlinks by default to efficiently store duplicated files but your machine does not support them in C:\Users\Work\.cache\huggingface\hub\models--bert-base-uncased. Caching files will still work but in a degraded version that might require more space on your disk. This warning can be disabled by setting the `HF\_HUB\_DISABLE\_SYMLINKS\_WARNING` environment variable. For more details, see https://huggingface.co/docs/huggingface_hub/how-to-cache#limitations.

To support symlinks on Windows, you either need to activate Developer Mode or to run Python as an administrator. In order to activate developer mode, see this article: <https://docs.microsoft.com/en-us/windows/apps/get-started/enable-your-device-for-development>

```
warnings.warn(message)
```

```
vocab.txt: 0%|          | 0.00/232k [00:00<?, ?B/s]
```

```
tokenizer.json: 0%|          | 0.00/466k [00:00<?, ?B/s]
```

```
config.json: 0%|          | 0.00/570 [00:00<?, ?B/s]
```

Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better performance, install the package with: `pip install huggingface\_hub[hf\_xet]` or `pip install hf\_xet`

```
model.safetensors: 0%|          | 0.00/440M [00:00<?, ?B/s]
```

Some weights of BertForMultipleChoice were not initialized from the model checkpoint at bert-base-uncased and are newly initialized: ['classifier.bias', 'classifier.weight']

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

In [28]: *# Training arguments*

```
training_args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=4,
    per_device_train_batch_size=2,
    logging_steps=1,
    save_steps=5,
    save_total_limit=1,
    remove_unused_columns=False,
)

# Data collator
def data_collator(features):
    batch_size = len(features)
    num_choices = features[0]["input_ids"].size(0)
    input_ids = torch.stack([f["input_ids"] for f in features]).view(batch_size, num_choices)
    attention_mask = torch.stack([f["attention_mask"] for f in features]).view(batch_size, num_choices)
    labels = torch.tensor([f["labels"] for f in features])
    return {
        "input_ids": input_ids,
        "attention_mask": attention_mask,
        "labels": labels,
    }

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset,
    data_collator=data_collator,
```

```
)  
  
# Train!  
trainer.train()
```



```

-----
ImportError                                Traceback (most recent call last)
Cell In[28], line 2
      1 # Training arguments
----> 2 training_args = TrainingArguments(
      3     output_dir=
      4     num_train_epochs=4,
      5     per_device_train_batch_size=2,
      6     logging_steps=1,
      7     save_steps=5,
      8     save_total_limit=1,
      9     remove_unused_columns=False,
     10 )
     12 # Data collator
     13 def data_collator(features):

File <string>:133, in __init__(self, output_dir, overwrite_output_dir, do_train, do_eval, do_predict, eval_strategy, prediction_loss_only, per_device_train_batch_size, per_device_eval_batch_size, per_gpu_train_batch_size, per_gpu_eval_batch_size, gradient_accumulation_steps, eval_accumulation_steps, eval_delay, torch_empty_cache_steps, learning_rate, weight_decay, adam_beta1, adam_beta2, adam_epsilon, max_grad_norm, num_train_epochs, max_steps, lr_scheduler_type, lr_scheduler_kwargs, warmup_ratio, warmup_steps, log_level, log_level_replica, log_on_each_node, logging_dir, logging_strategy, logging_first_step, logging_steps, logging_nan_inf_filter, save_strategy, save_steps, save_total_limit, save_safetensors, save_on_each_node, save_only_model, restore_callback_states_from_checkpoint, no_cuda, use_cpu, use_mps_device, seed, data_seed, jit_mode_eval, use_ipex, bf16, fp16, fp16_opt_level, half_precision_backend, bf16_full_eval, fp16_full_eval, tf32, local_rank, ddp_backend, tpu_num_cores, tpu_metrics_debug, debug, dataloader_drop_last, eval_steps, dataloader_num_workers, dataloader_prefetch_factor, past_index, run_name, disable_tqdm, remove_unused_columns, label_names, load_best_model_at_end, metric_for_best_model, greater_is_better, ignore_data_skip, fsdp, fsdp_min_num_params, fsdp_config, fsdp_transformer_layer_cls_to_wrap, accelerator_config, deepspeed, label_smoothing_factor, optim, optim_args, adafactor, group_by_length, length_column_name, report_to, ddp_find_unused_parameters, ddp_bucket_cap_mb, ddp_broadcast_buffers, dataloader_pin_memory, dataloader_persistent_workers, skip_memory_metrics, use_legacy_prediction_loop, push_to_hub, resume_from_checkpoint, hub_model_id, hub_strategy, hub_token, hub_private_repo, hub_always_push, hub_revision, gradient_checkpointing, gradient_checkpointing_kwargs, include_inputs_for_metrics, include_for_metrics, eval_do_concat_batches, fp16_backend, push_to_hub_model_id, push_to_hub_organization, push_to_hub_token, mp_parameters, auto_find_batch_size, full_determinism, torchdynamo, ray_scope, ddp_timeout, torch_compile, torch_compile_backend, torch_compile_mode, include_tokens_per_second, include_num_input_tokens_seen, neftune_noise_alpha, optim_target_modules, batch_eval_metrics, eval_on_start, use liger_kernel, liger_kernel_config, eval_use_gather_object, average_tokens_across_devices)

File c:\Users\Work\anaconda3\envs\explainable_ai\Lib\site-packages\transformers\training_args.py:1795, in TrainingArguments.__post_init__(self)
     1793 # Initialize device before we proceed
     1794 if self.framework == "pt" and is_torch_available():
-> 1795     self.device
     1797 # Disable average tokens when using single device
     1798 if self.average_tokens_across_devices:

File c:\Users\Work\anaconda3\envs\explainable_ai\Lib\site-packages\transformers\training_args.py:2324, in TrainingArguments.device(self)

```

```

2320 """
2321 The device used by this process.
2322 """
2323 requires_backends(self, ["torch"])
-> 2324 return self._setup_devices

```

File c:\Users\Work\anaconda3\envs\explainable_ai\Lib\site-packages\transformers\utils\generic.py:74, in cached_property.__get__(self, obj, objtype)

```

72 cached = getattr(obj, attr, None)
73 if cached is None:
---> 74     cached = self.fget(obj)
75     setattr(obj, attr, cached)
76 return cached

```

File c:\Users\Work\anaconda3\envs\explainable_ai\Lib\site-packages\transformers\training_args.py:2194, in TrainingArguments._setup_devices(self)

```

2192 if not is_sagemaker_mp_enabled():
2193     if not is_accelerate_available():
-> 2194         raise ImportError(
2195             f"Using the `Trainer` with `PyTorch` requires `accelerate>={ACCE
LERATE_MIN_VERSION}`: "
2196             f"Please run `pip install transformers[torch]` or `pip install
'accelerate>={ACCELERATE_MIN_VERSION}`"
2197         )
2198 # We delay the init of `PartialState` to the end for clarity
2199 accelerator_state_kwargs: dict[str, Any] = {"enabled": True, "use_configured
_state": False}

```

ImportError: Using the `Trainer` with `PyTorch` requires `accelerate>=0.26.0`: Please run `pip install transformers[torch]` or `pip install 'accelerate>=0.26.0'`

In []: